WIF3004 SOFTWARE ARCHITECTURE AND DESIGN PARADIGMS

Lecture 1

Do not upload any materials (teaching materials such as lecture notes; questions and sample answers of tutorials/labs/assignments/projects/tests/quizzes/exams/etc.; audio/video recordings or photos/screen shots of live or online lectures/tutorials/labs/chats/presentation /etc.; and your solutions, etc.) of this course anywhere online due to copyright and privacy. You can upload your solutions to Spectrum. Thank you for your cooperation.

Outline

- What is designing?
- Design Thinking
- Software development tasks as ill-structured problems (ISPs)

The City Car Club (CCC)

- CCC is a car-sharing system that will be operating within a city.

The City Car Club (CCC) – The specification

The *City Car Club* (CCC) is an organisation that is planning to set up a car-sharing system that will eventually operate in different cities, and requires a software system to manage this. They will be trying out this idea for the first time in ⟨*your favourite city*⟩. CCC will provide a number of small 'city-friendly' cars that can be used by members of the club, who pay a monthly membership fee. When using a car, the pick-up and drop-off points must lie within a specified central zone of the city, although a car may go outside this area while it is in use. There will be some designated locations where cars can

The City Car Club (CCC) – The specification

area while it is in use. There will be some designated locations where cars can be left and collected, indicated by specially coloured hatching on the parking bays, and with these being protected by a lockable 'fold-down' post where it is necessary to stop others from using them. Cars may also be left in any other suitable parking area, including a number of supermarket car parks (approval has been obtained for this), but if a car is left in any other non-designated places, the club member who is leaving the car will be personally responsible for any parking fees or fines that might be incurred. A member will also be charged an additional fee if a car is left outside of the designated central zone.

The City Car Club (CCC) – The specification

The position and status of the cars is tracked using a GPS system built into each car. At any time each car will have a status, describing its availability, which can have one of the following values:

- *Unavailable* – the car is currently in use by a member of the club, or has been identified by the support team as needing attention in some way.
- *Available* – the car is available for use and can be booked by a club member.
- *Reserved* – a member has requested use of the car, but has not yet taken charge of it.

CCC's central monitoring system will continuously record and track the position and status of each car.

The City Car Club (CCC) – The specification

To use a car (a 'session'), a member uses their club id to access the central booking system, identifies a car that is available as well as conveniently positioned for their use, and makes a reservation, with a unique confirmation code being sent to them as an SMS text or in an e-mail. The reservation can be made using a laptop application or a phone app (these are provided when a member registers with CCC). A reservation automatically expires if a car has not been collected within 20 minutes of making the reservation, but it can be extended for one additional period of 20 minutes (the system prompts the member at the end of the first 20-minute period and gives them the opportu-

The City Car Club (CCC) – The specification

member at the end of the first 20-minute period and gives them the opportunity to extend it). On reaching the car they have reserved, the member uses the unique code to open the car and to operate the ignition system, as well as to unlock the security post if one is in use. This can all be done by using either a small keypad that is provided to members by CCC, or by means of a Bluetooth link to the app on their phone or tablet. Entering the code starts the session, and from that point on the member is responsible for the car until they finish the session by locking the car and notifying the central booking system.

The City Car Club (CCC) – The specification

Billing for a session is based upon a formula that takes account of distance covered and the length of time the car is being used. Members can also add fuel to a car at a number of authorised agencies (including the supermarkets providing car parking) using a payment card that charges the fuel to CCC, for which they get credit. There is a 'fine' for leaving a car with an inadequate reserve of fuel in the tank (the fuel level is transmitted to the central monitoring system at the start and end of a session). As well as identifying where nearby available cars are located, the phone app provided by CCC can be used to locate nearby filling stations.

The City Car Club (CCC) – The specification

The support team of CCC can perform various 'mobile' tasks of maintenance, including fixing any problems with cars and retrieving any that have been left outside of the central area. While out 'on the road', they will use a similar set of apps to those provided for ordinary members, but with additional privileges (for example, allowing them to declare a car to be unavailable when they begin work on it, and available once they have finished).

In the longer term, once the system has been tested out in this first location, CCC plan to make their cars available in other cities. So the design of their software needs to ensure that this is adaptable for use in other places, in addition to the usual requirements for security, robustness and efficiency.

What is Designing About?

- What is designing in the context of software development?

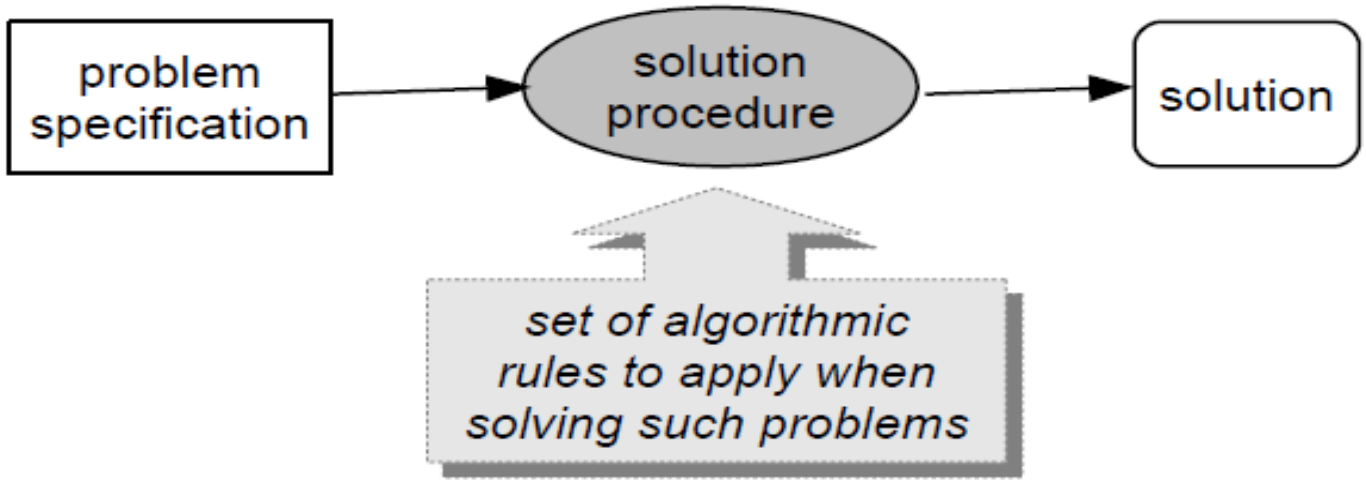
When is design needed?

- Two types of problem (Herb Simon 1973):
  - **Well-structured problem (WSP)**
    - A 'right' solution
    - Clear tests that the right solution has been found.
    - E.g. $x^2 - 5x + 6=0$; $x = 2$ and $x = 3$
    - E.g. many problems in science, esp. physics and maths
  - **Ill-structured problem (ISP)**
    - No explicit formulation and many possible solutions (good or bad, rather than right or wrong)
    - No definitive test to determine correctness of a result, so cannot know that the 'best' solution has been achieved ('no stopping rule').
    - E.g. many problems addressed by engineers

When is design needed?

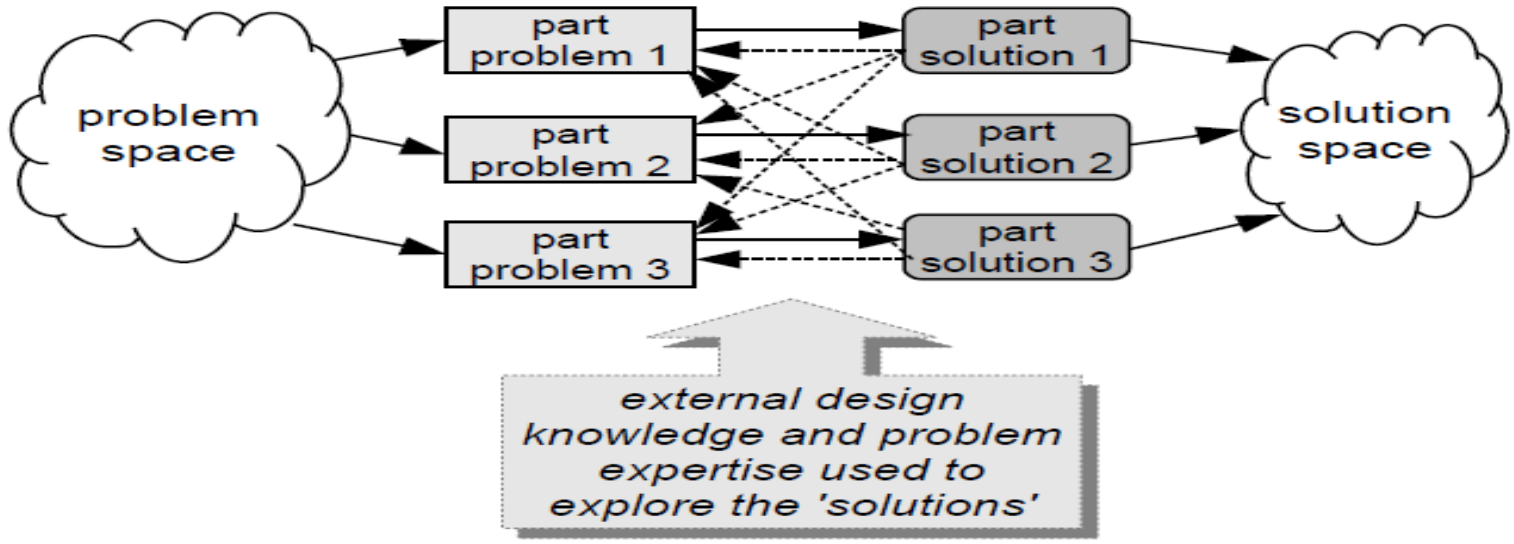
- **Question**: Writing program or develop software is WSP or ISP?

When is design needed?



a) Process for 'solving' a WSP

When is design needed?



b) Process for 'solving' an ISP

- Solving one part of the problem might influence our understanding of that part and our ideas for solving other parts of the problem

When is design needed?

The CCC system as an ISP

Let's consider a design scenario that might occur when the software design team start to plan how they will develop the software for CCC.

1. The head designer suggests representing each car as an object, with its current status being stored internally, and a set of methods that allow customers and other users (such as the maintenance team and the accounting software) to access that status.

2. The next question is how to organise all of these objects. So a member of the team suggests using an array of objects. One consequence is that every car now has to have an index number so that the relevant object can be accessed.

3. And then there's the question of how to model the status of a car internally (is it available, unavailable or reserved?).

None of these decisions exists in isolation, and each one has implications for the way that the design will evolve.

When is design needed?

- ISPs – require the use of some form of *design process* to formulate a possible solution and then evaluate how well it might meet a particular need

So, when is design needed?

- Your answer:

Characteristics of Software

- Pre-internet era
  - Two main types of software: system software (OS), application
- Internet era (early 1990s)
  - new ideas about how to structure software (classes, objects, services)
  - interact with applications running on remote computers
- Subsequently
  - Different parts can be hosted on cloud, tablets, phones, domestics appliances (IoT)

Characteristics of Software

- Need to evolve (previously described as maintenance)
- Technical debt
  - Refers to the future consequences that may arise as a consequence of the current decisions that we make about software development.
  - Is technical debt good or bad?
  - Student software projects versus real software projects.

Characteristics of Software

- Technical debt (continue) (Lenarduzzi et al., 2021)
  - "Technical Debt (TD) is a metaphor introduced by Cunningham(1992) to represent sub-optimal design or implementation solutions that yield a benefit in the short term but make changes more costly or even impossible in the medium to long term (Avgeriou et al., 2016b)."
  - Have to manage such sub-optimal solutions.
  - Just like financial debt, a TD has an interest attached.

Characteristics of Software

- Technical debt (continue) (Lenarduzzi et al., 2021)
  - TD is inevitable and even desirable under some circumstances.
  - Current best practices: Try to avoid it or refactoring or rewriting code and other artifacts to remove the accumulated sub-optimal solutions and their negative impact.

Characteristics of Software

- Technical debt (continue)
  - Some examples of technical debts [1]:
    - Poor software code quality
    - No documentation
    - Insufficient software testing

[1] https://www.techtarget.com/searchitoperations/tip/6-technical-debt-examples-and-how-to-solve-them

Characteristics of Software

The CCC and technical debt

Let's consider some possible scenarios for how the business of CCC might evolve and some possible effects that these may have upon the design decisions outlined earlier.

- If the scheme proves highly successful, CCC may need to expand their fleet of cars. *Consequence*: the early decision to use an array of objects might prove inconvenient, although this does depend upon how it was implemented.
- Some customers may need larger cars (even though this is a 'city' club!). *Consequence*: while this can be addressed by using inheritance to create new subclasses of the 'car' class, doing so may create other problems, such as how to meet the need for billing for use according to car size.
- CCC may conclude that they need a more elaborate set of status values, for example: "available but outside of the central zone". *Consequence*: this may complicate or extend the set of methods for each object, depending on how these were specified.

None of these is insurmountable, but it emphasises the point that design decisions, especially those made at early stages, may have consequences for later.

What exactly do we mean by ‘design’?

- Two meanings: as a noun and as a verb
- First meaning (as a noun)
  - as a *noun* that describes something that forms a *plan* of how we intend some artefact to be constructed.
  - a ‘design plan’ – to provide a ‘model’ of how some form of ‘product’ should be constructed.
  - Two forms of design plan
    - A design plan for a physical artefact (E.g. a car.)
    - A design plan for a procedure or process (E.g. a musical score, a recipe).
    - Can combine the two forms of design plan (E.g. design plan for a rack may describe both the structure of the rack and specify the way it should be assembled)

What exactly do we mean by ‘design’?

- Second meaning (as a verb)
  - As a verb, as in ‘to design’
  - The process of designing something – with the goal to produce a design plan.

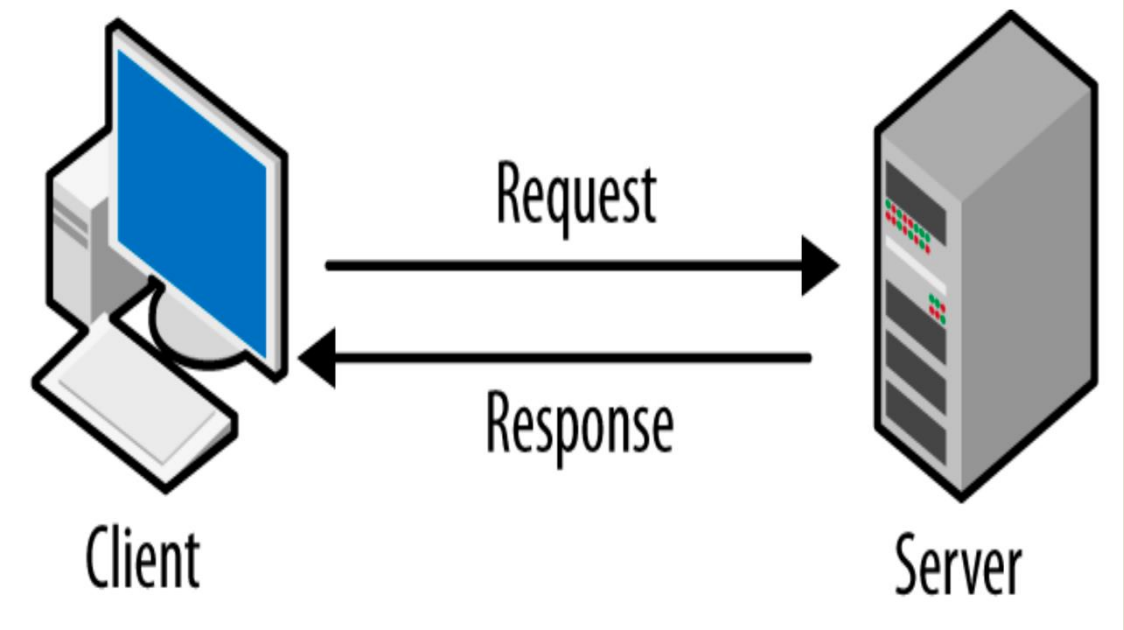
What exactly do we mean by ‘design’?

- Combining the two meanings….

Design as verb creating design as noun

Design Thinking

- Refers to thinking about how to design something
- 3 perspectives:
  - Design as styling
    - how some form of artefact is styled
    - 'look and feel' of software
  - Design as planning
    - plan the activities involved in a software development project
    - E.g. create software from 'components'
  - **Design as adaptation**
    - New ways of doing things
    - E.g. adapt 'design pattern', 'architecture style/pattern' to address current need
    - Main perspective in software development, but all 3 perspectives can be seen in many examples.



- Client-server architecture pattern

Design Thinking

Design perspectives and the CCC

Designing the software for the CCC will mainly be about design as *adaptation*, bringing different forms together to create the models and structures required to meet the needs of CCC. However it will also involve some design as *styling* when considering things like user interfaces that need to be easy to use on a mobile phone. Design as *planning* will mainly be concerned with how the development process is organised—for example whether or not we employ an *agile* approach to development, although there will be some interaction with the first form when considering how the business might develop and expand.

Designing is creative

- When undertaking design activities, our objective is to create something that does not already exist, which involves predicting an effect upon the future that will only be realised if the design plan is feasible. To do so, we need to be able to envisage how something will be done, and then work backwards to devise ways of bringing this about.

Take-home points

- ISP
- Technical debt
- Design as a noun
- Design as a verb

References

- Budgen, D. 2020. Software Design – Creating Solutions for Ill-Structured Problems. 3rd Edition, CRC Press.
- Lenarduzzi, V., Besker, T., Taibi, D., Martini , A., Fontana, F. A. 2021. *A systematic literature review on Technical Debt prioritization: Strategies, processes, factors, and tools*, Journal of Systems and Software, Volume 171,2021,110827, ISSN 0164-1212, https://doi.org/10.1016/j.jss.2020.110827.